

Thesis

Vehicle-in-the-Loop simulation using CARLA simulator

Author:

Kamolov Makhmud



Budapest University of Technology and Economics
Department of Control for Transportation and Vehicle Systems

Supervisor:

Szőke László

Thesis:

May 08, 2026

Contents

1 Introduction

1.1 What is autonomous driving	3
1.2 Importance of Simulation in Autonomous Driving	4
1.3 State of the Art in Autonomous Driving Simulators	6
1.4 ADAS Applications	10
1.5 Related Works	13

2 Interface Co-Simulation Design

2.1 Selecting the Suitable Simulator	16
2.2 Modeling the Car in the Environment	16
2.3 Data Gathering with Autonomous Driving Mode	21
2.4 Data Gathering with Manual Driving Mode	23

3 Lane Keeping Assist Implementation in CARLA

3.1 Co-Simulation Architecture	26
3.2 Lateral Control: Theoretical Background	28
3.3 Lane-Shift Estimation from Waypoints	30
3.4 PID Controller Implementation	33
3.5 Vehicle and Sensor Initialisation	34
3.6 Manual Driver Override	36
3.7 The Main Control Loop	39
3.8 Reference: CARLA's Built-in Local Planner	40

A Carla and Pycharm IDE Interfacing

Abstract

Abstract will be added after completion of the thesis.

Chapter 1

Introduction

1.1 What is autonomous driving

For 3 million years human being has been creating tools to ease their daily chores in order to make better life. One of the successful creation of human as decent invention is vehicles. First automobile ever created is involved to late 19th century. Since that vehicles have been evolving rapidly, so, recent decades we had new concept of vehicle so-called “Autonomous Vehicle”.

Autonomous Vehicles (AVs), or self-driving cars, are vehicles as discussed earlier that can ease human daily driving tasks by taking them one by one to do it themselves.

Autonomous vehicles represent the next major step in the evolution of road transportation, extending earlier advances in vehicle automation and driver assistance systems. As sensing, computing power, and control technologies have matured, vehicles have gained the ability to monitor their surroundings, interpret complex traffic situations, and support or even replace human driving tasks. Early prototypes in the 1980s and 1990s, enabled by progress in computer vision, robotics, and embedded systems, demonstrated that vehicles could follow lanes, maintain speed, and avoid obstacles under constrained conditions. The parallel development of GPS-based positioning, digital mapping, and onboard sensors such as cameras, radars, and lidars provided the foundation for modern perception systems. Last 7 decades, Autonomous driving has got big advances which can be seen in Figure 1.1. These technological advances collectively enabled the transition from basic driver assistance toward higher levels of automation, where vehicles can execute steering, acceleration, and braking tasks with increasing independence from human input. Driverless vehicle technology gained momentum at the beginning of the 21st century. Initiatives like Google’s driverless car project [1] captured public attention and created a general awareness that autonomous driving was a real possibility. The use of cameras, radars, lidars, and artificial intelligence algorithms in vehicles endowed them with the capability to navigate complex traffic situations independently. Many car manufacturers and technology companies are now accelerating their efforts to develop fully autonomous vehicles. The goal of these vehicles is to reduce traffic accidents, increase transportation efficiency, and provide greater independence for people with mobility limitations. However, technical challenges, as well as ethical, legal, and safety issues, continue to shape the advancement in this field. The future of autonomous driving depends on overcoming these challenges and gaining societal acceptance for this new technology.

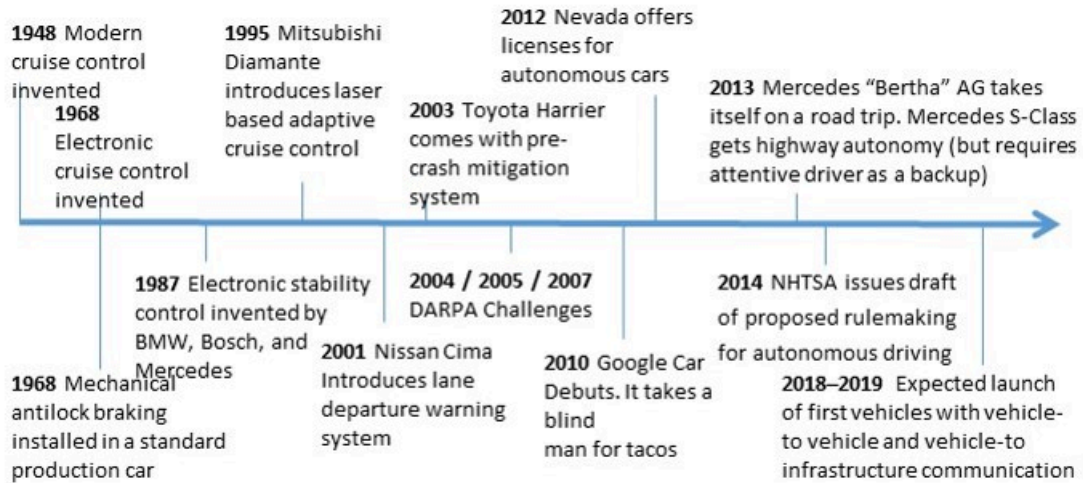


Figure 1.1: The historical development of autonomous driving [2].

1.2 Importance of Simulation in Autonomous Driving

In today's world, developing and using software has become very important across many industries. While these software solutions provide us with numerous conveniences and advantages, they also introduce certain requirements. One of the most critical requirements for any software is testing. The testing phase can be used to determine whether an application fulfills its intended function or to assess its efficiency. Of course, in some sectors, this functionality takes on an even greater importance. One such area is safety. In industries like aviation and automotive, where a user's safety can be directly affected by an error in the application, testing and simulation are more important than ever. In model-based software applications, after a model is created, it moves to the verification stage. Before being loaded onto the hardware, the model undergoes several verification steps (as V model in Figure 1.2). Some of these verifications are named Model In the Loop (MIL) [3], Software In the Loop (SIL) [4], and Hardware In the Loop (HIL) [5].

Model in the Loop (MIL) In the initial stages of system design, before real hardware or software components are implemented, the simulation and testing of system models are carried out by the Model in the Loop (MIL) testing method. This method uses mathematical models and simulations for system or component design. These models are utilized to understand how the designed system will function and behave. MIL testing is a critical tool for verifying the design and identifying potential problems at an early stage. With this method, engineers and designers can gain valuable insights into how the system will operate before actually manufacturing the hardware or fully developing the software.

Software in the Loop (SIL) The Software in the Loop (SIL) testing method is the process of testing a system or component's software without using real hardware.

This method simulates environmental factors or other parts interacting with the system while the software runs directly on a computer. This simulation evaluates the software's behavior and performance under real-world conditions. The main goal of SIL testing is to verify the software's functionality, detect errors at an early stage, and analyze interactions between the system and the software. This approach accelerates the software development and refinement processes and prevents costly errors.

Hardware in the Loop (HIL) The Hardware in the Loop (HIL) testing method creates an environment to control and test real hardware. In this method, the hardware being tested, such as a vehicle control unit, operates in real-time, but the software simulates the physical systems (e.g., engines, sensors, actuators, etc.) interacting with the hardware in real-time. This simulation allows for the evaluation of how the hardware performs under real-world conditions.

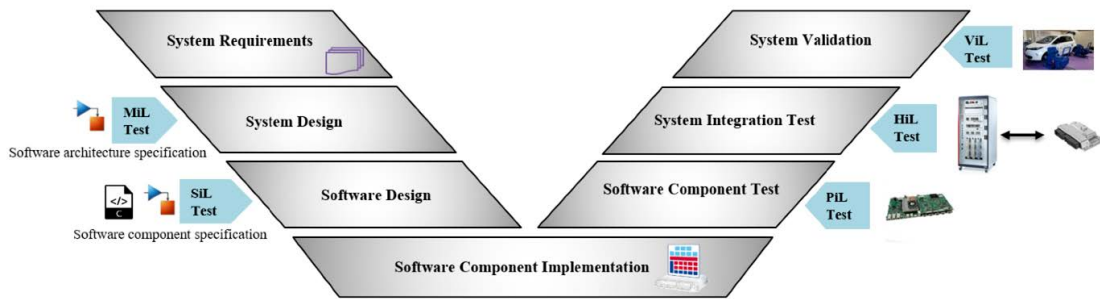


Figure 1.2: MIL, SIL , PIL, HIL and VIL tests in V-cycle development process [6].

In Figure 1.3 driving test simulator is shown. Test simulators typically fall under the SIL category. The Software in the Loop (SIL) testing method has numerous advantages. These benefits explain why SIL tests are an important part of the software development process and demonstrate their value to engineers and developers across various industries. The main advantages can be listed as follows:

Cost Efficiency: SIL tests are conducted in a simulated environment that does not require real hardware, thus reducing the costs associated with purchasing, maintaining, and repairing hardware. Additionally, early diagnosis of potential errors helps prevent more costly problems later on.

Rapid Feedback Loop: Testing software in a model allows for quick iterations in the development process. This enables developers to instantly see the effects of their code and make fast modifications if necessary.

Risk Reduction: Tests conducted without real hardware use a simulated environment to safely examine situations that could be potentially dangerous or harmful to the hardware. This is especially important when expensive hardware is involved.

Broad Test Scenarios: SIL tests are capable of quickly and easily testing a wide range of scenarios that mimic real-world conditions. Developers can experiment with system parameters, error states, and various operational conditions.

Acceleration of the Development Process: SIL tests can speed up the development process instead of waiting for the hardware to be ready. Testing the software alongside hardware development shortens the time to market for the product.

Preparation for Integration and System Tests: When software is successfully tested in a SIL environment, the transition to more complex testing stages where hardware and software are run together becomes easier. This helps reduce problems that may arise during system tests and integration.

Flexibility in the Development Process: Software can be tested and developed with various hardware platforms and configurations. This flexibility allows for the evaluation of how the software will perform on different systems.



Figure 1.3: Driving Test Simulator [7]

1.3 State of the Art in Autonomous Driving Simulators

Advancements in the field of autonomous driving have also led to the development of powerful autonomous driving simulators. Nowadays, a wide range of simulators produced by various companies are in use. These include open-source projects (e.g. CARLA [8], AirSim [9]), commercial software, and customized simulation solutions used in academic and industrial research. In the context of this thesis, a simulation environment is essential, because the Lane Keeping Assist (LKA) function is developed and evaluated entirely in a virtual Vehicle-in-the-Loop setup rather than on a physical test vehicle. Therefore, it is crucial to clearly define the requirements that the simulator must satisfy—such as realistic sensor models, controllable traffic and weather, and reproducible scenarios—and to justify the choice of a specific platform.

In the following, I first review several popular autonomous driving simulation tools and platforms available in the literature and in practice. The comparison focuses on criteria such as graphic quality, accuracy of the physics engine, sensor simulation capabilities,

simulation of traffic and pedestrians, weather conditions, and the ability to simulate at different times of the day. Based on these criteria, I motivate the selection of CARLA as the main simulation platform for this work and explain how it will be used to implement and test the proposed LKA function.

1.3.1 Waymo simulator



Figure 1.4: Comparing Waymo Simulator graphics and the real world.

Waymo possesses an advanced simulation platform [10] and is widely recognized as a leader in autonomous vehicle technology. This simulator plays a crucial role in the development and testing of autonomous driving systems by providing a highly realistic (can be seen in Figure 1.4) and comprehensive virtual environment. It combines high-fidelity graphics that accurately replicate real-world environments with a precise physics engine capable of modeling vehicle dynamics, collisions, and surface interactions. Furthermore, it includes detailed sensor emulation for LIDAR, radar, and cameras, enabling a realistic representation of environmental perception. The platform can also model complex traffic flow and pedestrian behaviors, which are essential for evaluating autonomous vehicles in dense urban settings. In addition, the simulator supports a wide range of weather and lighting conditions, allowing performance assessment under rain, snow, fog, or low-light scenarios. Owing to these capabilities, the Waymo simulator holds a significant position among state-of-the-art autonomous driving simulation platforms.

1.3.2 SVL Simulator



Figure 1.5: SVL Simulator Screen

The autonomous vehicle research community frequently employs the LGSVL Simulator [11], an open-source platform designed for high-fidelity testing of perception and control algorithms. Thanks to its Unity-based rendering engine, LGSVL offers realistic visual environments suitable for evaluating camera-based functions. Its physics engine provides detailed modeling of vehicle dynamics, tire-road interactions, and collisions, enabling credible closed-loop testing of control algorithms. The simulator includes configurable sensor models—such as lidar, radar, and cameras—along with comprehensive traffic and pedestrian behavior, allowing researchers to model complex urban scenarios (Figure 1.5). Furthermore, LGSVL supports diverse weather and lighting conditions, which makes it valuable for assessing the robustness of autonomous driving functions under varying environmental circumstances. LGSVL represents an important reference point when comparing available simulation platforms.

1.3.3 Sim4CV

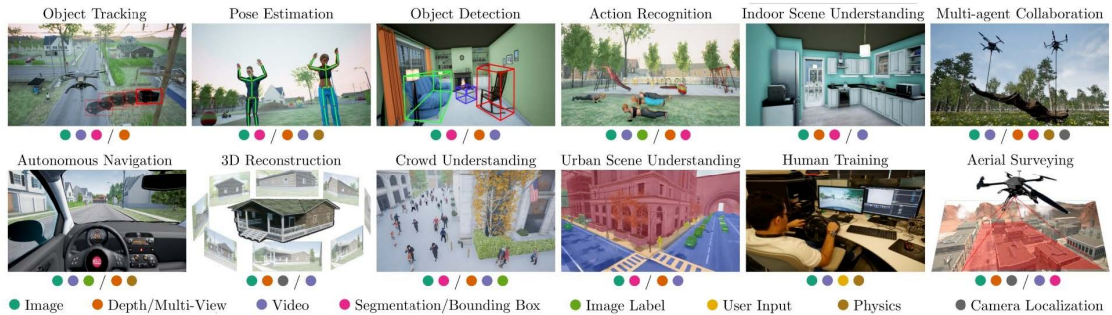


Figure 1.6: Sim4CV: A Photo-Realistic Simulator for Computer Vision Applications

Sim4CV is a simulation platform developed for research in computer vision and autonomous systems, with a strong emphasis on autonomous driving and aerial navigation [12]. Built on Unreal Engine 4, it provides high-quality visual environments and a

physics engine capable of modeling realistic vehicle dynamics, object interactions, and environmental effects. The platform supports the simulation of various sensors, including cameras and lidar, enabling detailed testing of perception algorithms. Sim4CV also offers traffic and pedestrian simulation, though the level of sophistication may vary across versions and configurations (Figure 1.6). Furthermore, the ability to adjust weather and lighting conditions makes it suitable for evaluating algorithm robustness under diverse environmental scenarios.

1.3.4 Carla Simulator



Figure 1.7: Carla Simulator Screen

The CARLA Simulator is an open-source autonomous driving platform [13] widely used in research for developing, testing, and validating autonomous vehicle algorithms. Built on Unreal Engine 4, it provides high-fidelity graphics that closely resemble real-world environments (Figure 1.7), which is essential for evaluating perception systems. CARLA’s underlying physics engine accurately models vehicle dynamics, collisions, and surface interactions, enabling realistic assessment of control and planning algorithms. It also offers comprehensive sensor simulation—including cameras, LIDAR, radar, and GNSS—allowing researchers to replicate and analyze diverse sensing conditions. In addition, CARLA supports dynamic traffic scenarios and human-like pedestrian behavior, creating complex and socially interactive environments for autonomous driving studies. Finally, its configurable weather and lighting system makes it possible to test algorithms across various environmental conditions, ensuring robustness and generalizability in real-world deployments.

Features / Simulators	Waymo Simulator	LGSVL Simulator	Sim4CV	CARLA Simulator
Graphics Quality	8/10	9/10	8/10	9/10
Physics Engine Accuracy	8/10	9/10	7/10	9/10
Sensor Simulation	8/10	9/10	7/10	9/10
Traffic and Pedestrian Simulation	8/10	9/10	6/10	9/10
Weather Conditions	7/10	8/10	6/10	9/10
Simulation at Different Times of Day	7/10	8/10	6/10	9/10

Table 1.1: Comparative Feature Scoring of Simulators

Using a set of predefined evaluation criteria—such as graphics quality, physics realism, sensor simulation, traffic modeling, and environmental variability—a comparative score was assigned to each simulator. Table 1.1 summarizes the evaluation results based on these criteria.

1.4 ADAS Applications

Advanced Driver Assistance Systems (ADAS) are becoming increasingly common in today’s vehicles. Developed to enhance vehicle safety and the driving experience, these systems utilize various technologies such as cameras, sensors, and radars to identify potential hazards and are designed to alert the driver or automatically control the vehicle under certain conditions. ADAS technologies not only assist drivers in traveling more safely and comfortably but also have the potential to reduce traffic accidents. There are various applications of ADAS, which are crucial for the safety of the driver and also provide significant conveniences. Some of these applications can be briefly discussed as follows:

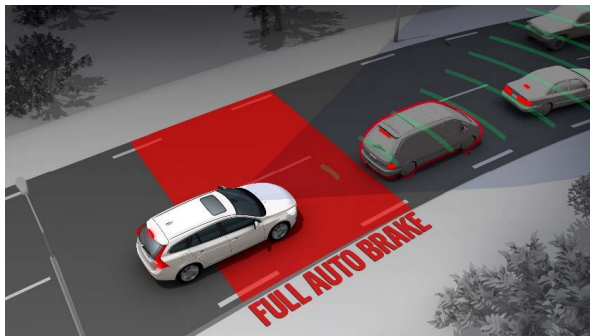


Figure 1.8: Automatic Emergency Braking system.

Automatic Emergency Braking (AEB):

The vehicle automatically brakes to avoid colliding with a vehicle or obstacles ahead [14]. A laser radar sensor detects the distance to the vehicle ahead and the relative speed. If there is an object in the area seen by the laser radar sensor at this speed, the brakes are activated to prevent a collision as shown in Figure 1.8. Additionally, in some vehicles, it works in conjunction with seat belts activated by braking before a

collision to help reduce injuries in cases where a collision is unavoidable.

Traffic Sign Recognition System: This system detects traffic signs and informs the driver about traffic rules such as speed limits and prohibition signs. Generally, an image is captured by a camera sensor placed on the vehicle’s windshield, and the detected image is projected onto the user’s screen (Figure 1.9). This technology aims to prevent accidents by ensuring that drivers do not overlook important warning signs on the road [15].



Figure 1.9: Traffic Sign Recognition System



Figure 1.10: Night Vision

20% of all road accidents, and up to 50% on certain roads, are fatigue-related [17]. Although the implementation of this system varies among manufacturers, some brands warn the driver by detecting steering movements and how often the vehicle drifts out of its lane.

Blind Spot Warning System: This system identifies the presence of other vehicles in the vehicle's blind spots and informs the driver about them. Vehicles in the blind spot are detected using radar sensors located on the sides of the rear bumper (Figure 1.11). A warning is displayed to the driver in the side mirror, and if the driver signals to change lanes while there is a vehicle in the blind spot, an alert is issued. In some models, changing lanes by braking is prevented to enhance safety of the driver in the car and to avoid road accident [18].

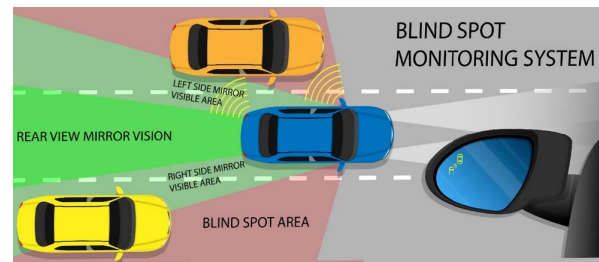


Figure 1.11: Blind Spot Warning System

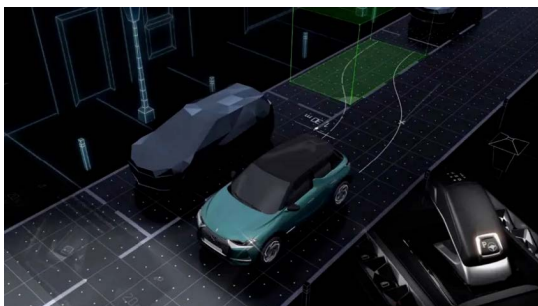


Figure 1.12: Parking Assistant

Night Vision: This system expands the driver's field of vision in low light conditions or at night through cameras or other sensors [16]. Utilizing thermal cameras and infrared lights, it detects living beings on the road ahead and provides audible or visual warnings to the driver (Figure 1.10). In addition to detection, advanced night vision systems can highlight potential hazards directly on the dashboard display or head-up display.

Fatigue Detection Systems: Various studies have suggested that approximately

Parking Assistant: This feature assists the driver in better positioning the vehicle while parking and, in some cases, can automatically park the vehicle (Figure 1.12). It utilizes parking sensors located around the vehicle to do this [19]. Parking sensors are generally electromagnetic sensors. Commonly, the system is used in a manner where the driver is still responsible for commands such as gas and gear changes.

Automatic Headlight Control: The effectiveness of vehicle headlights becomes even more crucial on roads with insufficient external lighting at night, such as forest roads. Oncoming drivers on these types of roads can be affected by the headlight beams. Systems like this reduce the light intensity when detecting an oncoming vehicle to minimize the adverse effects on the opposite driver. In more advanced models, the direction of the headlight beam is adjusted to achieve this effect (Figure 1.13).

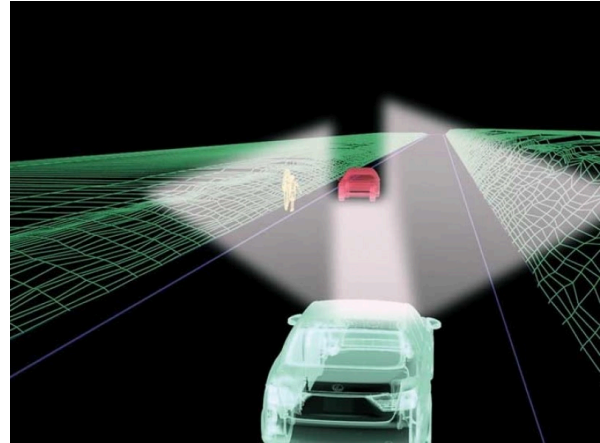


Figure 1.13: Automatic Headlight Control



Figure 1.14: Lane Keeping Assistant

Lane Keeping Assistant: This system uses a camera sensor to detect the distance of the vehicle from the lane markings and activates when the determined distance falls below a certain threshold (Figure 1.14). It steers the steering wheel to direct the vehicle back into the lane, ensuring it remains within its lane boundaries. It acts as a countermeasure to unintentional drifting caused, reducing relevant accidents by over 20%.

Adaptive Cruise Control (ACC): This system is a more advanced version of the traditional cruise control system [20]. While a traditional cruise control maintains the vehicle at a set speed, this system adapts the speed based on the speed of the vehicle ahead (Figure 1.15). It utilizes two separate sensors to accomplish this. The first sensor is a radar sensor that measures the distance to the vehicle in front. The second sensor is a speed sensor that detects any decrease or increase in the vehicle's speed. Based on the information from these sensors, decisions are made and implemented to accelerate or decelerate. Generally, the desired distance and following speed can be adjusted by the user via controls on the steering wheel.

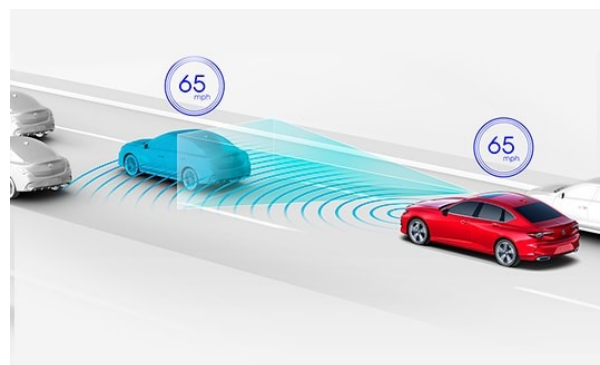


Figure 1.15: Adaptive Cruise Control

1.5 Related Works

The research community has proposed various methods for validating autonomous driving functions using simulation and Vehicle-in-the-Loop (ViL) concepts, as well as a wide range of lateral control strategies for path and lane tracking. This section summarises the most relevant works and explains how they relate to the present thesis, which implements a Lane Keeping Assist (LKA) function in the CARLA simulator using an external Python-based controller and an RGB camera.

Vehicle-in-the-Loop simulation

Son et al. propose a PG-Based Vehicle-In-the-Loop Simulation (VILS) framework that combines a real vehicle with a virtual driving environment for system development and consistency validation [21]. Their system includes modules for virtual road generation, synchronisation between real and virtual domains, a virtual traffic manager, and perception sensor modelling. They demonstrate that the behaviour observed in the VILS setup is consistent with physical vehicle tests over different speeds, road types and surrounding environments. This work shows how ViL can improve reproducibility and safety when testing autonomous driving systems.

Cheng et al. provide a comprehensive survey on testbench-based Vehicle-in-the-Loop simulation testing for autonomous vehicles [22]. They review existing autonomous vehicle testing approaches and identify limitations of traditional methods. The paper then focuses on testbench-based VIL architectures, principles and equipment, with particular emphasis on physical signal stimulation and realistic road condition simulation. The survey identifies research gaps between current testbench technologies and industrial AV testing needs, and highlights ViL as a promising tool for accurate and efficient validation.

Xiong Hu et al. present a ViL simulation test framework for autonomous vehicles, where an autonomous driving system is evaluated using a virtual environment combined with real vehicle components [23]. Their work emphasises the synchronisation of vehicle dynamics and simulated scenarios and discusses how ViL can be used to test autonomous driving functions safely and systematically.

These works clearly show that ViL is an important methodology for validating autonomous driving systems. However, they mainly focus on general architectures, testbench setups, and system-level validation. They do not specifically address a software-only ViL configuration in a driving simulator like CARLA, where a simulated vehicle is controlled by an external code using RGB camera images. Furthermore, the detailed design and evaluation of a single ADAS function such as Lane Keeping Assist—implemented and tested entirely in such a ViL setup—remains less explored. This is where the present thesis contributes.

Lateral control for autonomous vehicles

Lateral control is a key component for lane keeping and path tracking. Kebbaty et al. provide a technical review of lateral control methods for autonomous wheeled vehicles

[24]. They classify control approaches into model-based and data-driven methods, and discuss their advantages, limitations and implementation challenges. The survey underlines that lateral control must deal with accuracy, robustness and real-time constraints, and highlights open challenges for future research.

Artuñedo et al. present a comparative evaluation of state-of-the-art lateral control strategies for autonomous vehicles [25]. They propose a systematic tuning methodology and define a comprehensive set of performance metrics covering tracking accuracy, robustness and comfort. The controllers are evaluated in extensive simulations and real-vehicle tests over different trajectories and scenarios. Their results show how different control strategies behave under the same conditions and provide guidance for selecting suitable lateral controllers.

Dominguez et al. experimentally compare several classical lateral controllers—such as Pure Pursuit and Stanley—on an autonomous vehicle platform [26]. They study the performance of these controllers in terms of tracking error and stability, using a combination of simulation and real-world experiments. Their work provides practical insight into the behaviour of commonly used path-tracking controllers and their suitability for autonomous driving applications.

Snider’s technical report is a widely cited reference on automatic steering methods for autonomous automobile path tracking [27]. It describes the mathematical formulation and implementation details of several steering controllers, including Pure Pursuit and Stanley, and discusses their application to autonomous vehicles. This report is frequently used as a theoretical foundation when implementing classical lateral control laws.

These studies provide a solid theoretical and experimental basis for the lateral control part of Lane Keeping Assist. However, most of them either assume ideal path or lane information (for example, predefined waypoints or ground-truth lane geometry) or focus on real-vehicle experiments without an explicit simulation-based ViL framework. They do not analyse in detail how classical lateral controllers perform when they receive lane information from a real-time camera-based perception pipeline inside a simulator.

Identified gap and relation to this thesis

In summary, the literature on ViL shows how combined real–virtual setups can be used to validate autonomous driving systems [21],[22],[23]}, while the lateral control literature provides a variety of controllers and comparative evaluations for path and lane tracking [24],[25],[26],[27]}. However, the combination of these two aspects

- a **Vehicle-in-the-Loop-style simulation environment** in a high-fidelity simulator, and
- a **camera-based Lane Keeping Assist function** implemented as an external control stack — is less represented in the existing works.

The present thesis addresses this gap by implementing a modular LKA system in CARLA, where an RGB camera provides input to an external Python program that estimates lane geometry and computes steering commands. The system is evaluated in a reproducible simulation environment, thus connecting the ideas of ViL-based validation and classical lateral control within a single, integrated Lane Keeping Assist framework.

Chapter 2

Interface Co-Simulation Design

2.1 Selecting the Suitable Simulator

The development and testing of autonomous driving technologies require a robust simulation environment. This environment must accurately model the real world, including vehicles, pedestrians, and various environmental conditions, while also providing comprehensive support for sensor simulation and enabling integration with analytical tools. After a detailed evaluation of existing simulators, including the Waymo Simulator, LGSVL Simulator, Sim4CV, and CARLA Simulator, based on critical features such as graphic quality, the accuracy of the physics engine, sensor simulation capabilities, the simulation of traffic and pedestrians, weather conditions, the ability to simulate different times of day, CARLA Simulator has been identified as the most suitable choice for my research objectives. CARLA provides high-quality graphics with Unreal Engine 4 [28] and its physics engine accurately models vehicle dynamics and environmental interactions, offering a solid foundation for testing autonomous driving algorithms under various conditions. Moreover, its ability to simulate a wide range of sensors used in autonomous vehicles, such as cameras, LIDAR, radar, and GNSS, with high fidelity is crucial for the development and testing of perception algorithms. CARLA also excels in simulating dynamic traffic scenarios and pedestrian behaviors, facilitating comprehensive testing of autonomous driving systems in complex urban environments. The capability to simulate different weather conditions and times of day is important for assessing the performance of autonomous vehicle systems under various environmental conditions. For my research, CARLA's Python API facilitates easy integration with an external controller, providing a seamless workflow for data analysis and algorithm testing. Consequently, CARLA Simulator's advanced graphics, accurate physics engine, extensive sensor simulation capabilities, and effective traffic and pedestrian simulation set it as the ideal choice for my autonomous driving research. Its compatibility with Python API further supports my analytical and development needs, making it the most suitable simulator for my project.

2.2 Modeling the Car in the Environment

In order to control a vehicle in the simulation, the controller must recognize the vehicle model. For this, a model of the vehicle is needed. Various models can be used when modeling the vehicle. Among these, the most commonly used ones usually offer a good

balance between simplicity and accuracy, capable of representing the vehicle's motion dynamics and control systems. In line with the needs of this project, the Dynamic Single-Track (DST) model has been chosen (shown in Figure 2.1). The DST model (bicycle model) can be modeled with single-track wheels (one front and one rear wheel). This is equivalent to a model where the right and left sides of a four-wheeled vehicle are considered equal.

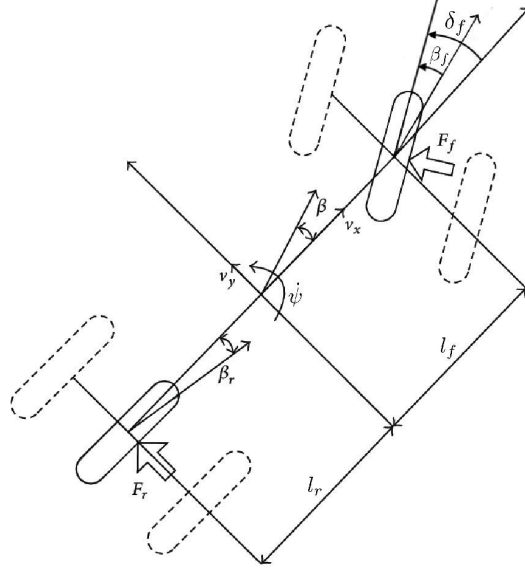


Figure 2.1: Single Track Model

Vehicle variables:

- δ_f : steering angle.
- β : vehicle sleep angle = angle between the vehicle longitudinal axis and velocity
- β_f, β_r : tire slip angles = angles between the tire longitudinal axis and velocity.

Vehicle parameters:

- CoG: center of gravity
- m, J : mass and moment of inertia
- l_f : distance CoG - front wheel center
- l_r : distance CoG - rear wheel center
- c_f, c_r : front/rear cornering stiffnesses.

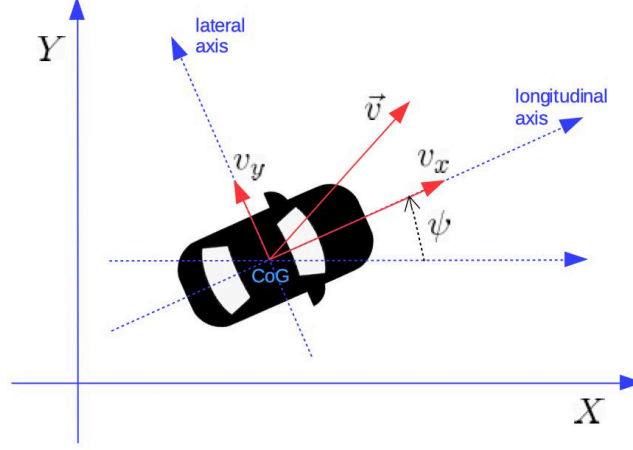


Figure 2.2: Vehicle Reference System

Vehicle Dynamic parameters:

- X, Y : coordinates of the vehicle CoG in an inertial reference frame (shown in Figure 2.2)
- ψ : yaw angle
- $\omega_\psi = \dot{\psi}$: yaw rate
- $\vec{v} \equiv V$: velocity vector in the inertial frame
- v_x : longitudinal speed = $\vec{v}$ component along the longitudinal axis
- v_y : lateral speed = $\vec{v}$ component along the lateral (transverse) axis
- a_x : longitudinal acceleration in the inertial frame.

The state equations of the DST model are:

$$\dot{X} = V_x \cos \psi - V_y \sin \psi \quad (2.1)$$

$$\dot{Y} = V_x \sin \psi + V_y \cos \psi \quad (2.2)$$

$$\dot{\psi} = \omega_\psi \quad (2.3)$$

$$\dot{V}_x = V_y \omega_\psi + a_x \quad (2.4)$$

$$\dot{V}_y = -V_x \omega_\psi + \frac{2}{m}(F_{yf} + F_{yr}) \quad (2.5)$$

$$\dot{\omega}_\psi = \frac{2}{J}(l_f F_{yf} - l_r F_{yr}) \quad (2.6)$$

where F_{yf} and F_{yr} are the lateral forces exchanged between tire and road.

For the tire model, there are several tire model options:

Linear (for $V_x = \text{const}$) tire model:

$$F_{yf} = -C_f \beta_f, \quad F_{yr} = -C_r \beta_r \quad (2.7)$$

$$\beta_f = \frac{V_y + l_f \omega_\psi}{V_x} - \delta_f, \quad \beta_r = \frac{V_y - l_r \omega_\psi}{V_x} \quad (2.8)$$

Nonlinear simplified tire model:

$$F_{yf} = -C_f \beta_f \cos \delta_f, \quad F_{yr} = -C_r \beta_r \quad (2.9)$$

$$\beta_f = \arctan\left(\frac{V_y + l_f \omega_\psi}{V_x}\right) - \delta_f, \quad \beta_r = \arctan\left(\frac{V_y - l_r \omega_\psi}{V_x}\right) \quad (2.10)$$

Nonlinear Pacejka's tire model:

$$F_{yf} = -f_p(\beta_f) \cos \delta_f, \quad F_{yr} = -f_p(\beta_r) \quad (2.11)$$

where β_f and β_r and $f_p(\beta)$ is given by the Pacejka's magic formula. **Pacejka's magic formula:**

$$f_p(\beta) = p_1 \sin(p_2 \arctan(p_3 \beta - p_4(p_3 \beta - \arctan(p_3 \beta)))) \quad (2.12)$$

p_1 : peak value, p_2 : shape factor, p_3 : stiffness factor, p_4 : curvature factor. Linearizing this formula, we find $p_1 p_2 p_3 = C_f$ (or C_r).

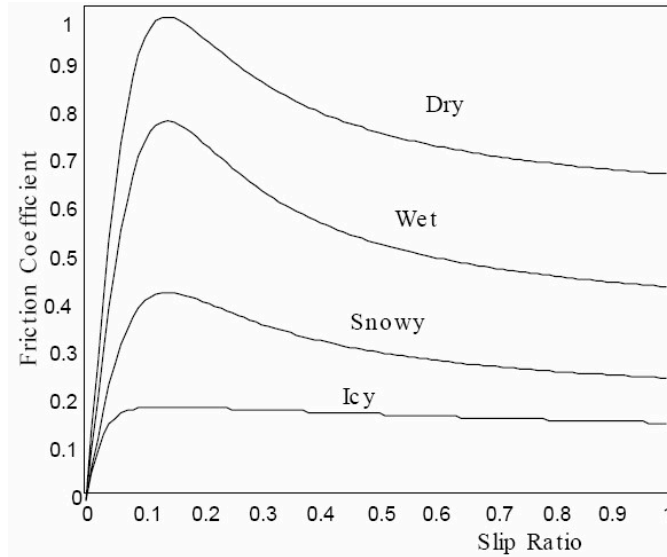


Figure 2.3: Friction Coefficients in different conditions

In the real world conditions, these parameters are really hard to measure or estimate because they change based on the road conditions (see Figure 2.3).

The real parameters of the vehicle in the simulation environment have been found with the help of Carla's `Vehicle.physics` command.

Vehicle Parameters Values

- m : 1318kg
- J : 2500kg/m²
- l_f : 1.54m
- l_r : 1.51m
- c_f : 15000N/rad
- c_r : 15000N/rad

2.2.1 Dispatching To Obtain Throttle-Brake Value From Acceleration Value

To control a vehicle in the Carla simulator, we need three pieces of information: steering angle, throttle, and brake commands. However, our model generates ax data. It is not possible to directly provide this data to Carla because this information only tells us about the acceleration or deceleration of the vehicle and the magnitude of these changes. Therefore, we need to scale this data to use it for throttle-brake information. To do this, we perform dispatching. In the context of control, “dispatching” usually refers to the process of distributing tasks or resources according to certain criteria or algorithms.

To accomplish this, we first need specific reference values. These reference values should be collected according to many different scenarios to ensure they are suitable for every scenario, not just one. In our case, data were collected for three different scenarios: when the vehicle completes a straight path in a sine wave pattern, when continuous braking and accelerating are performed on a straight road, and finally, when only accelerating is done and the vehicle slows down without braking due to its own weight. The data include the vehicle's ax acceleration value, and throttle and brake values. By examining these accelerations, the priorities of throttle and brake values are determined for acceleration and deceleration situations. For instance, it is not always necessary to brake for negative acceleration values; reducing the throttle can also achieve the necessary speed reduction. Therefore, each value has its weight. Also, there is a range within which each command should be applied. These ranges can be determined using an if/else structure based on the acceleration value. The commands to be applied within these ranges are found by dividing the ax value by certain weight values. Consequently, the throttle and

brake values are instantaneously derived from the ax value and provided to the vehicle in the Carla environment for application. As explained above, these values were found by analyzing reference data and through trial and error. The code for the if/else structure that creates the intervals for applying these values and commands is provided below.

2.3 Data Gathering with Autonomous Driving Mode

A reference point was selected from the Carla Map to collect reference data. The vehicle was spawned at this reference point. A simulation duration was determined based on a finish point we had predetermined. The vehicle was moved using Carla's autonomous driving mode. During this time, the necessary reference position, waypoint data, yaw angle, and speed data were recorded into a .csv file at intervals of $T_s = 0.05$ seconds. Once the required simulation duration was completed, the vehicle was removed from the map.

```

1
2  ...
3
4  def carla_data_collector(vehicle: carla.Vehicle):
5      time = carla.Timestamp.frame
6      transform = vehicle.get_transform()
7      location = transform.location
8      location_dictionary = {"x": location.x, "y": location.y, "z":
location.z}
9      rotation = transform.rotation
10     rotation_dictionary = {"pitch": rotation.pitch, "yaw":
rotation.yaw, "roll": rotation.roll}
11     velocity = vehicle.get_velocity()
12     velocity_dictionary = {"x": velocity.x, "y": velocity.y, "z":
velocity.z}
13     acceleration = vehicle.get_acceleration()
14     acceleration_dictionary = {"x": acceleration.x, "y":
acceleration.y, "z": acceleration.z}
15     angular_velocity = vehicle.get_angular_velocity()
16     angular_velocity_dictionary = {"x": angular_velocity.x, "y":
angular_velocity.y, "z": angular_velocity.z}
17     vehicle_world = vehicle.get_world()
18     waypoint = vehicle_world.get_map().get_waypoint(location)
19     waypoint_dictionary = {"x": waypoint.transform.location.x, "y":
waypoint.transform.location.y, "z": waypoint.transform.location.z}

```

```

20     all_data = {"time": time, "location": location_dictionary, "ro-
rotation": rotation_dictionary, "velocity": velocity_dictionary, "ac-
celeration": acceleration_dictionary, "angular_velocity":
angular_velocity_dictionary, "waypoint": waypoint_dictionary}
21     return all_data
22
23 def carla_data_collector_2(vehicle: carla.Vehicle):
24     world = vehicle.get_world()
25     frame = world.get_snapshot().frame
26     waypoint =
vehicle.get_world().get_map().get_waypoint(vehicle.get_transform().lo-
cation)
27
28     waypoint_location = waypoint.transform.location
29     waypoint_forward_vector =
waypoint.transform.get_forward_vector()
30     vector_x = waypoint_forward_vector.x
31     vector_y = waypoint_forward_vector.y
32     waypoint_dictionary = {"x": waypoint_location.x, "y":
waypoint_location.y, "vector_x": vector_x, "vector_y": vector_y}
33     return waypoint_dictionary
34
35     ...
36
37 data = {
38     "waypoint_x": [],
39     "waypoint_y": [],
40     "vector_x": [],
41     "vector_y": []
42 }
43
44 ...
45
46 all_data = carla_data_collector_2(ego_vehicle)
47     if not data["waypoint_x"] and not data["waypoint_y"]:
48         data["waypoint_x"].append(all_data["x"])
49         data["waypoint_y"].append(all_data["y"])
50         data["vector_x"].append(all_data["vector_x"])
51         data["vector_y"].append(all_data["vector_y"])
52     else:

```



```

53         if all_data["x"] != data["waypoint_x"][-1] and
           all_data["y"] != data["waypoint_y"][-1]:
54             data["waypoint_x"].append(all_data["x"])
55             data["waypoint_y"].append(all_data["y"])
56             data["vector_x"].append(all_data["vector_x"])
57             data["vector_y"].append(all_data["vector_y"])

```

2.4 Data Gathering with Manual Driving Mode

The first capability that we demonstrate with CARLA is localization, which allows our ego-vehicle to determine its pose in the world. Two coordinate frames: the map frame, which is a coordinate frame that is fixed at the initial position of the map, and the vehicle frame, which is a coordinate frame attached to the middle of the rear axle of the vehicle. For our particular experiment, we record the vehicle's accurate pose while traversing a curved-straight route in the Town10. In the Python API, there is a class that comprises all the localization information for an actor at a certain moment in time; its methods comprise Getters such as `get_acceleration`, `get_velocity`, `get_transform`, and `get_angular_velocity`- which in our case we utilize the `get_transform` method which includes both location of the object (X , Y , Z from the origin of the map) in meters, and its rotation characteristics from which we utilize the yaw values. More concisely, X and Y coordinates were our focus, as in the map chosen, the road is entirely flat. Hence only these two coordinates remain crucial for tracking the vehicle's trajectory. Also, the Yaw angle, describing the orientation of the map's coordinate system, is essential for indication of the direction the vehicle is facing. Therefore for curved paths, which our scenarios contain, it is important.

Some additional considerations also are to be noted, such as the sampling rate, which determines the time step of the data collection. We have chosen 0.1, indicating not too sparse to miss some critical dynamics, especially for sharp turns, and also not too frequent leading to redundant information.

After initializing the scenario and ego vehicle (based on the preferences), based on the duration of the scenario depending on the controls of the car (throttle, brake, and steering commands), we record the data of the vehicle at each time step; for better understanding the following Script used in the development is provided;

```

1
2  import carla
3  import pandas as pd
4
5  # A dictionary that accumulates one row of data per simulation step
6  data = {

```

```

7     "frame": [], "x": [], "y": [], "yaw": [],
8     "v_x": [], "v_y": [], "a_x": [],
9     "throttle": [], "brake": [], "steer": []
10 }
11
12 while not crashed:
13     world.tick()                # Advance the synchronous
    simulation
14     snapshot = world.get_snapshot()
15     transform = ego_vehicle.get_transform()
16     velocity = ego_vehicle.get_velocity()
17     acceleration = ego_vehicle.get_acceleration()
18     control = ego_vehicle.get_control()
19
20     data["frame"].append(snapshot.frame)
21     data["x"].append(transform.location.x)
22     data["y"].append(transform.location.y)
23     data["yaw"].append(transform.rotation.yaw)
24     data["v_x"].append(velocity.x)
25     data["v_y"].append(velocity.y)
26     data["a_x"].append(acceleration.x)
27     data["throttle"].append(control.throttle)
28     data["brake"].append(control.brake)
29     data["steer"].append(control.steer)
30
31     # ... PyGame rendering and event handling ...
32
33     pd.DataFrame(data).to_csv("manual_drive_log.csv", index=False)

```

In manual driving mode, the autopilot is disengaged with `vehicle.set_autopilot(False)` and the user is given full control of throttle, brake, and steering through the keyboard. Each pressed key is captured as a PyGame event and is translated into a `carla.VehicleControl` object that is then applied to the ego vehicle in the simulator. While the user drives, the same getters introduced for the autonomous mode are queried at every simulation tick, so that the resulting `.csv` file shares the same structure for both modes and can be loaded by the controller without any additional preprocessing.

This dual-mode acquisition serves two purposes. First, it provides reference trajectories for the dispatcher tuning and for the controller design, since the autonomous mode of CARLA produces well-formed paths along the center of the lane. Second, it allows the recording of edge cases that are hard to reproduce automatically—for example aggressive

lane changes, harsh braking, or following curves at the limit of stability—because they can be performed deliberately by a human driver. The two `.csv` files are then used as the input data set for the lane keeping assist controller described in Chapter 3.

Chapter 3

Lane Keeping Assist Implementation in CARLA

The previous chapter established the simulation environment, the vehicle model and the data acquisition pipeline. This chapter describes how those building blocks are combined into a working Lane Keeping Assist (LKA) function. The implementation is centred on a single Python program, `pygame_controller.py`, which connects to the CARLA server, spawns the ego vehicle and the surrounding traffic, attaches an RGB camera, computes the lateral offset of the vehicle with respect to the centre of the lane, and feeds this offset to a discrete-time PID controller that returns a steering command. The same program also exposes a manual override through the keyboard so that the driver can take control at any moment. A reference implementation provided by the CARLA framework, `local_planner.py`, is discussed at the end of the chapter to position the proposed controller relative to the standard library tooling.

3.1 Co-Simulation Architecture

The Lane Keeping Assist function is implemented as an external client that communicates with the CARLA server through the Python API. The server, running the Unreal Engine renderer and the physics integration, is responsible for the propagation of the world state at a fixed time step. The client retrieves the relevant fraction of that state—the pose of the ego vehicle, the geometry of the closest lane, and the latest camera frame—computes a control command, and sends it back to the server before the next tick is requested. Conceptually this is the same pattern used in any Vehicle-in-the-Loop setup, with the difference that here the “vehicle” is itself a simulated actor rather than a physical platform.

Three design choices are essential to make this loop reproducible:

Synchronous mode. The simulator is configured so that the world only advances when the client explicitly calls `world.tick()`. Without this setting, the server would tick at its own rate and the controller would observe a non-deterministic stream of measurements. The corresponding configuration in the implementation is reported below.

```
1 settings = world.get_settings()
2 settings.synchronous_mode = True
```

```

3  settings.no_rendering_mode = True    # rendering is delegated to
    PyGame
4  settings.fixed_delta_seconds = 0.05 # 20 Hz simulation rate
5  world.apply_settings(settings)
6
7  traffic_manager = client.get_trafficmanager()
8  traffic_manager.set_synchronous_mode(True)
9  traffic_manager.set_random_device_seed(0)

```

A fixed step of $T_s = 0.05$ s (20 Hz) is small enough to capture the closed-loop dynamics of the steering channel—whose bandwidth on a passenger car rarely exceeds a few hertz—and large enough not to overload the network connection between the client and the server.

Deterministic traffic. The Traffic Manager is also placed in synchronous mode and seeded with a fixed value. Each surrounding vehicle is then spawned at a randomly chosen spawn point and given to the Traffic Manager through `set_autopilot(True)`. To create a slightly more challenging environment, a random fraction of the vehicles is allowed to ignore traffic lights through `traffic_manager.ignore_lights_percentage`. Because the random seed is fixed, the same scenario can be replayed an arbitrary number of times—an essential property for tuning the controller and for comparing two configurations of the same algorithm.

External rendering. The third setting, `no_rendering_mode = True`, disables the in-engine spectator window. The visual feedback is instead drawn by PyGame, using the frames produced by an RGB camera attached to the ego vehicle. This decoupling has two advantages: first, the engine no longer needs to render the third-person spectator view, which reduces the load on the GPU; second, the same callback that produces the on-screen image can in principle be replaced or extended with image-processing routines, which keeps the door open for a perception-based version of the controller.

The resulting control loop is sketched in Figure 3.1. At every tick, four operations are executed in sequence: the world is advanced, the ego state and the lane reference are queried, the lateral controller produces a steering command, and the manual-override block decides whether to apply that command or to forward the keyboard inputs of the driver instead.

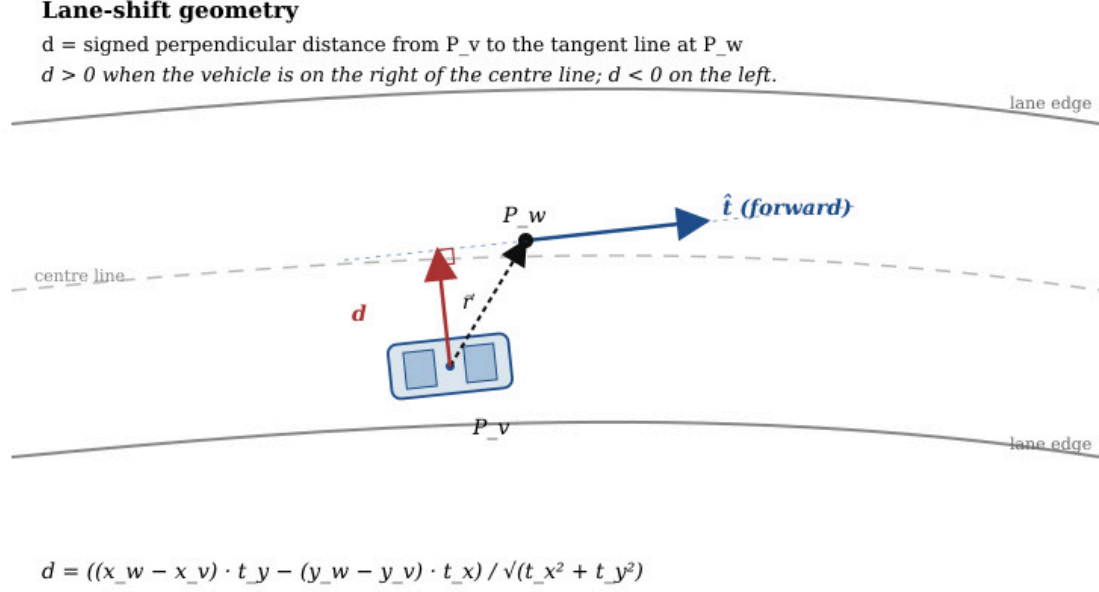


Figure 3.1: Block diagram of the proposed Lane Keeping Assist co-simulation. The CARLA server holds the world state; the client computes the steering command from the lane reference and the vehicle pose, and applies it through `vehicle.apply_control`.

3.2 Lateral Control: Theoretical Background

Lateral control is the part of the driving task that decides *how much to steer*. From the point of view of the lane keeping problem, the goal is to drive a scalar error signal—the lateral offset of the vehicle with respect to the centre of the lane—to zero. Several control laws are commonly used for this purpose; geometric trackers such as Pure Pursuit [27] and Stanley [26] derive a steering angle directly from a look-ahead point on the reference path, while feedback-based formulations close the loop on the offset itself. A comparative discussion of these strategies is given in the lateral-control review by Kebbati et al. [24] and in the experimental study of Artuñedo et al. [25]. In the present work the controller is realised with a Proportional–Integral–Derivative (PID) feedback law because of its simplicity, the low number of parameters to tune, and the clear interpretation of each term in the context of lane keeping.

3.2.1 The Continuous-Time PID Law

Let $e(t)$ denote the lateral error of the vehicle with respect to the centre of the lane at time t . A PID controller produces a steering-related command $u(t)$ as the linear combination of three terms:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.13)$$

Each term has an intuitive role in the LKA context. The proportional term $K_p e(t)$ produces a steering action proportional to the present offset and is responsible for the immediate reaction of the controller. The integral term $K_i \int e(\tau) d\tau$ accumulates the past error and removes the steady-state bias that may appear, for instance, on a constantly cambered road or under a small wheel-alignment offset. The derivative term $K_d \dot{e}(t)$ anticipates the future evolution of the error from its rate of change and adds damping when the vehicle is approaching the centre of the lane.

The qualitative effect of changing each gain is summarised in Table 3.2. These dependencies are well known in the control literature; they are repeated here only because they directly inform the manual tuning that was carried out during the implementation.

Gain	Rise time	Overshoot	Settling time	Steady-state error
Increase K_p	decreases	increases	small change	decreases
Increase K_i	decreases	increases	increases	eliminates
Increase K_d	small change	decreases	decreases	no effect

Table 3.2: Qualitative effect of each PID gain on the closed-loop response of a generic plant.

3.2.2 Discrete-Time Implementation

The controller runs at the same frequency as the simulator, that is at $T_s = 0.05$ s. The continuous-time integral and derivative therefore have to be approximated. Using the backward-Euler rule for the integral and a first-order backward difference for the derivative, the PID law becomes

$$I[k] = I[k-1] + e[k]T_s \quad (3.14)$$

$$D[k] = \frac{e[k] - e[k-1]}{T_s} \quad (3.15)$$

$$u[k] = K_p e[k] + K_i I[k] + K_d D[k] \quad (3.16)$$

The discrete formulation introduces two practical issues that have to be addressed in the implementation. The first one is *integral wind-up*. When the steering command saturates at the physical limit of the actuator, the integral term keeps growing as long as the lateral error does not change sign. Once the vehicle finally crosses the centre of the lane, the controller has to “discharge” this accumulated integral before it can produce a counter-steering action, which leads to a long undershoot. The simplest remedy, used in the implementation, is to clamp $I[k]$ to a symmetric interval $[-I_{\max}, +I_{\max}]$ at every step.

The second issue is *output saturation* proper. The CARLA `VehicleControl.steer` field accepts values in $[-1, +1]$, with -1 corresponding to a fully left and $+1$ to a fully right steering wheel. Any control law must therefore saturate its output to this range; in the implementation the same saturation is applied symmetrically with $u_{\max} = 1.0$.

3.2.3 Choice of the Sampling Period

The choice $T_s = 0.05$ s deserves a comment. From the point of view of stability of the discrete-time PID, T_s should be at least one order of magnitude smaller than the dominant time constant of the closed-loop system. For a passenger car at moderate speed the lateral dynamics has a time constant in the order of half a second, so any T_s below 50 ms is acceptable. From the point of view of the simulator, smaller steps inflate the wall-clock time of an experiment without producing additional information, since the world geometry and the traffic do not change appreciably below $T_s = 0.02$ s. The choice of 20 Hz is therefore a good compromise between accuracy and runtime.

3.3 Lane-Shift Estimation from Waypoints

The PID law described in the previous section assumes that the lateral error $e[k]$ is available at every step. In a perception-based pipeline this signal would be reconstructed from the camera image by detecting the lane markings; in the present implementation it is computed from the waypoint graph that the CARLA map exposes through its Python API. The choice of using waypoints rather than image processing is a deliberate one: it isolates the lateral controller from the perception layer, so that the closed-loop behaviour can be studied independently from the limitations of any specific lane-detection algorithm.

3.3.1 The CARLA Waypoint Graph

A CARLA map is internally represented as a directed graph whose nodes are *waypoints*. Each waypoint sits exactly on the centre line of a lane and stores the local pose of that lane, including the position (x_w, y_w, z_w) , a forward unit vector $\hat{t} = (t_x, t_y)$ tangent to the lane in the direction of legal traffic flow, and a number of metadata fields (lane width, lane change permissions, junction flags). For a vehicle at world position (x_v, y_v) , the call

```
1 waypoint = world.get_map().get_waypoint(vehicle.get_transform().location)
```

returns the closest waypoint on the closest drivable lane. This is the primitive that the data-gathering routine `carla_data_collector_2`, defined in the previous chapter, uses to record a reference trajectory: at every tick of the autonomous-driving session, the position of the closest waypoint and its forward vector are written to the `.csv` file. The full content of the resulting file is therefore a discrete sampling of the centre line of the lane along the recorded path, expressed in the same map coordinate frame as the vehicle pose.

3.3.2 Geometric Computation of the Lateral Offset

Given a waypoint $P_w = (x_w, y_w)$ with forward unit vector $\hat{t} = (t_x, t_y)$, and the current vehicle position $P_v = (x_v, y_v)$, the signed perpendicular distance from P_v to the line through P_w in the direction of $\hat{t}$ is the lateral offset of the vehicle with respect to the centre of the lane at that point. Denoting by $\vec{r} = P_w - P_v = (x_w - x_v, y_w - y_v)$ the position vector from the car to the waypoint, the offset is the scalar projection of $\vec{r}$ on the unit normal $\hat{n} = (-t_y, t_x)$:

$$d = \vec{r} \cdot \hat{n} = -(x_w - x_v)t_y + (y_w - y_v)t_x \quad (3.17)$$

By rearranging the signs and dividing by the norm of $\hat{t}$, which protects the formula in the case where the forward vector returned by the API is not exactly normalised, the expression used in the code is recovered:

$$\text{lane_shift} = \frac{(x_w - x_v)t_y - (y_w - y_v)t_x}{\sqrt{t_x^2 + t_y^2}} \quad (3.18)$$

Figure 3.2 illustrates the construction. The sign of the result is positive when the vehicle is on the right of the lane centre, with respect to the direction of the forward vector $\hat{t}$, and negative when it is on the left; a sign convention that the PID controller can interpret directly as “steer right” versus “steer left”.

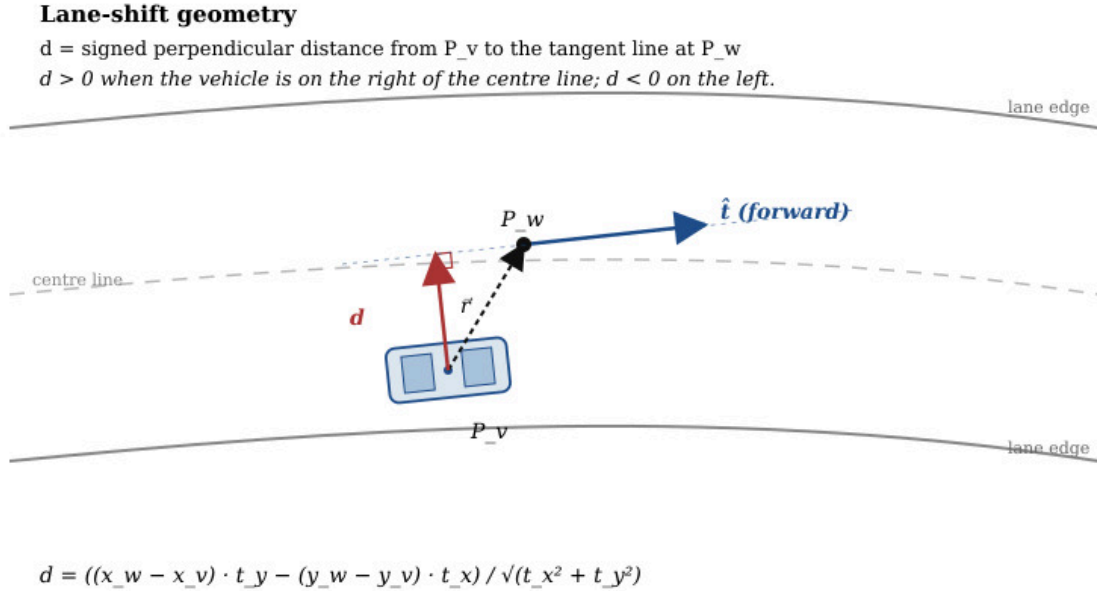


Figure 3.2: Geometric definition of the lane shift d as the signed perpendicular distance between the vehicle position P_v and the line through the closest waypoint P_w in the direction of the lane forward vector $\hat{t}$.

3.3.3 Look-Ahead Search Strategy

The reference .csv file contains several thousand waypoints, while the vehicle only moves a few metres per tick. Recomputing the closest waypoint by an exhaustive search at every step would therefore be wasteful and, more importantly, would expose the controller to ambiguities on closed circuits, where two waypoints may be geometrically close but topologically distant. The implementation uses instead a *local* search anchored at the index reached at the previous step. The corresponding routine, `lane_shift_calculator`, is reported below.

```
1  index = 0
2  def lane_shift_calculator(vehicle: carla.Vehicle, data:
    pd.DataFrame):
3      global index
4
5      vehicle_location = vehicle.get_transform().location
6      car_x, car_y = vehicle_location.x, vehicle_location.y
7
8      nearest = float('inf')
9      nearest_index = index
10
11     future_horizon = 3
12     for i in range(future_horizon + 1):
13         idx = index + i
14         if idx >= len(data):
15             break
16         wp_x = data.iloc[idx]["waypoint_x"]
17         wp_y = data.iloc[idx]["waypoint_y"]
18         distance = np.sqrt((car_x - wp_x)**2 + (car_y - wp_y)**2)
19         if distance < nearest:
20             nearest = distance
21             nearest_index = idx
22
23     index = nearest_index
24     vector_x = data.iloc[nearest_index]["vector_x"]
25     vector_y = data.iloc[nearest_index]["vector_y"]
26     wp_x = data.iloc[nearest_index]["waypoint_x"]
27     wp_y = data.iloc[nearest_index]["waypoint_y"]
28     lane_shift = ((wp_x - car_x) * vector_y - (wp_y - car_y) *
        vector_x) \
29         / np.sqrt(vector_x**2 + vector_y**2)
```

```
30     return lane_shift
```

Two design choices are worth highlighting. First, the search horizon `future_horizon = 3` only inspects the current waypoint and the next three. This window is wide enough to absorb the few centimetres that the vehicle covers in one tick at the speeds used during the experiments, but narrow enough to remain $O(1)$ in the length of the reference path. Second, the search index is updated *monotonically*: once the nearest waypoint has moved forward in the file, it is never allowed to move backward. This convention prevents the controller from “snapping” to a previous lap on a closed circuit, but it also means that the function must be re-initialised when a new reference trajectory is loaded.

When the local search reaches the end of the file (`idx >= len(data)`), the loop is broken and the last valid index is reused. In a production-quality version of the controller this condition would trigger a graceful disengagement of the LKA function; in the present implementation, it simply causes the lateral error to remain frozen at the last available waypoint, which is acceptable as long as the recorded trajectory is longer than the experiment.

3.4 PID Controller Implementation

The discrete-time PID law of the previous section is implemented as a stateful function that holds the integral and the previous error in module-level variables. The full implementation is reported below.

```
1  integral = 0.0
2  prev_error = 0.0
3  I_max = 1.0
4  u_max = 1.0
5
6  def pid_controller(lane_shift, Kp=1.0, Ki=0.0, Kd=0.0):
7      global integral, prev_error, I_max, u_max
8
9      dt = 0.05 # 50 ms
10
11     # Integral with anti-windup clamp
12     integral += lane_shift * dt
13     integral = max(min(integral, I_max), -I_max)
14
15     # Backward-difference derivative
16     derivative = (lane_shift - prev_error) / dt
17
18     # PID combination
```

```

19     u = Kp * lane_shift + Ki * integral + Kd * derivative
20
21     # Output saturation to the CARLA steer range
22     u = max(min(u, u_max), -u_max)
23
24     prev_error = lane_shift
25     return u

```

The function is purposely kept short, but a few details are worth a comment.

State variables. The integral and the previous error are stored at module level rather than as attributes of an object. This is sufficient for a single-vehicle controller and makes the function easy to call from the main loop, but it also means that the controller cannot be instantiated twice in the same process without explicit reset. A future refactor could wrap the same logic in a class to remove this limitation.

Sampling time. The constant `dt = 0.05` is duplicated with the `fixed_delta_seconds` setting of the simulator. Keeping a single source of truth would prevent the two values from drifting if the simulation rate is ever changed; in the current code base, this synchronisation is enforced by convention rather than by construction.

Anti-windup. The clamping of the integral to $[-I_{\max}, +I_{\max}]$ implements the simplest form of anti-windup. More sophisticated schemes—for instance the back-calculation method, in which the difference between the saturated and the unsaturated control action is fed back into the integrator—were not necessary at the speeds used in the experiments, where the steering command rarely saturates.

Default gains. The default values $K_p = 1.0$, $K_i = 0.0$, $K_d = 0.0$ correspond to a pure proportional controller, which is a safe starting point for tuning. The values used at the call site of the main loop, $K_p = 1.0$, $K_i = 0.01$, $K_d = 0.1$, were obtained by manual tuning on a straight section of the recorded trajectory and then refined on the curved sections; their robustness across different scenarios is a topic for the experimental chapter.

3.5 Vehicle and Sensor Initialisation

The control loop relies on three actors that are spawned before the simulation starts: a fleet of background vehicles that populate the map, the ego vehicle that is controlled by the LKA function, and an RGB camera that is attached to the ego vehicle and provides the visual feedback for the operator.

3.5.1 Background Traffic

The background traffic is generated by sampling the available spawn points of the map and placing a randomly chosen vehicle blueprint at each of them. To keep the population realistic, the blueprint library is filtered down to a list of consumer-grade models:

```

1  models = ['dodge', 'audi', 'model3', 'mini', 'mustang', 'lincoln',
2           'prius', 'nissan', 'crown', 'impala']
3  blueprints = []
4  for vehicle in world.get_blueprint_library().filter('*vehicle*'):
5      if any(model in vehicle.id for model in models):
6          blueprints.append(vehicle)
7
8  max_vehicles = min(50, len(spawn_points))
9  vehicles = []
10 for spawn_point in random.sample(spawn_points, max_vehicles):
11     actor = world.try_spawn_actor(random.choice(blueprints),
12     spawn_point)
13     if actor is not None:
14         vehicles.append(actor)
15
16 for vehicle in vehicles:
17     vehicle.set_autopilot(True)
18     traffic_manager.ignore_lights_percentage(vehicle,
19     random.randint(0, 50))

```

Two safety measures are embedded in this code. The use of `try_spawn_actor` rather than `spawn_actor` prevents the program from raising an exception when two spawn points are too close to each other; if the collision check of the simulator fails, the corresponding slot is silently skipped. The cap at `max_vehicles = 50` prevents the population from outgrowing the number of available spawn points on smaller maps. Finally, after each vehicle is spawned, the Traffic Manager is asked to occasionally ignore traffic lights, which adds a controlled amount of irregularity to the scenario.

3.5.2 The Ego Vehicle and the Camera

The ego vehicle is then chosen at random among the vehicles that were successfully spawned. An RGB camera is attached to it through `world.spawn_actor`, with a `Rigid` attachment so that the camera follows the body of the vehicle without any compliance:

```

1  ego_vehicle = random.choice(vehicles)
2
3  bound_x = 0.5 + ego_vehicle.bounding_box.extent.x
4  bound_y = 0.5 + ego_vehicle.bounding_box.extent.y
5  bound_z = 0.5 + ego_vehicle.bounding_box.extent.z
6
7  camera_init_trans = carla.Transform(carla.Location(x=+0.8*bound_x,
8  y=+0.0*bound_y,
9  z=+1.3*bound_z))

```

```

10 camera_bp = world.get_blueprint_library().find('sensor.camera.rgb')
11 camera_bp.set_attribute("image_size_x", "1280")
12 camera_bp.set_attribute("image_size_y", "720")
13 camera = world.spawn_actor(camera_bp, camera_init_trans,
14                             attach_to=ego_vehicle,
15
16                             attachment_type=carla.AttachmentType.Rigid)
16 camera.listen(lambda image: pygame_callback(image, renderObject))

```

The camera transform is expressed as multiples of the half-extent of the vehicle bounding box, so that the same code adapts automatically to vehicles of different size. With $0.8b_x$ along the longitudinal axis and $1.3b_z$ along the vertical, the camera is mounted approximately at the position of the rear-view mirror, looking forward—a placement that mimics a windshield-mounted ADAS camera.

The image stream is consumed by `pygame_callback`, a small function whose only responsibility is to reshape the raw byte array delivered by CARLA into a $H \times W \times 3$ RGB array and to convert it into a PyGame surface that can be blitted on the screen. The reshape order is dictated by the BGRA layout used internally by the simulator; the slicing `[:, :, :-1]` swaps the colour channels and the slice `[:, :, :3]` discards the alpha channel.

```

1 def pygame_callback(data, obj):
2     img = np.reshape(np.copy(data.raw_data), (data.height,
3         data.width, 4))
4     img = img[:, :, :3]
5     img = img[:, :, :-1]
6     obj.surface = pygame.surfarray.make_surface(img.swapaxes(0, 1))

```

In this implementation the camera is used only as a visual feedback device for the human operator. The lateral controller does not consume the camera frames. This separation is deliberate, as discussed in the introduction of Section 3, and it is what makes the architecture *open* to a future perception-based extension: a lane-detection block can be inserted between the camera callback and the controller without modifying any of the surrounding code.

3.6 Manual Driver Override

A Lane Keeping Assist function is, by definition, a driver-assistance system: the driver must be able to take control of the vehicle at any moment. In the implementation this requirement is realised by a `ControlObject` class that stores the desired throttle, brake and steering values and applies them to the vehicle through a single `apply_control` call.

```

1 class ControlObject(object):

```

```

2     def __init__(self, veh):
3         self._vehicle = veh
4         self._steer = 0
5         self._throttle = False
6         self._brake = False
7         self._steer_cache = 0
8         self._control = carla.VehicleControl()
9
10    def parse_control(self, event):
11        if event.type == pygame.KEYDOWN:
12            if event.key == pygame.K_RETURN:
13                self._vehicle.set_autopilot(False)
14                if event.key == pygame.K_UP:         self._throttle = True
15                if event.key == pygame.K_DOWN:       self._brake = True
16                if event.key == pygame.K_RIGHT:      self._steer = 1
17                if event.key == pygame.K_LEFT:       self._steer = -1
18            if event.type == pygame.KEYUP:
19                if event.key == pygame.K_UP:         self._throttle = False
20                if event.key == pygame.K_DOWN:       self._brake = False
21                if event.key in (pygame.K_LEFT, pygame.K_RIGHT):
22                    self._steer = None

```

The class follows a two-step design that is common in PyGame applications. The `parse_control` method is called for every keyboard event and only updates the internal flags; the heavier `process_control` method is called once per simulation tick and turns those flags into actual `VehicleControl` values. This separation prevents the simulator from receiving a burst of nearly identical commands when the user holds a key down, and produces a smoother steering response by applying small, time-integrated increments to the steering value:

```

1     def process_control(self):
2         # Throttle / brake combinations
3         if self._throttle:
4             self._control.throttle = min(self._control.throttle +
5             0.01, 1)
6             self._control.gear = 1
7             self._control.brake = False
8         elif not self._brake:
9             self._control.throttle = 0.0
10        if self._brake:
11            if self._vehicle.get_velocity().length() < 0.01 and not
12                self._control.reverse:

```

```

11         self._control.brake = 0.0
12         self._control.gear = 1
13         self._control.reverse = True
14         self._control.throttle = min(self._control.throttle
+ 0.1, 1)
15     elif self._control.reverse:
16         self._control.throttle = min(self._control.throttle
+ 0.1, 1)
17     else:
18         self._control.throttle = 0.0
19         self._control.brake = min(self._control.brake + 0.3,
1)
20     else:
21         self._control.brake = 0.0
22
23     # Steering with low-pass return-to-centre
24     if self._steer is not None:
25         self._steer_cache += 0.03 * self._steer
26         self._steer_cache = max(-0.7, min(0.7,
self._steer_cache))
27         self._control.steer = round(self._steer_cache, 1)
28     else:
29         self._steer_cache *= 0.2
30         if abs(self._steer_cache) < 0.01:
31             self._steer_cache = 0.0
32         self._control.steer = round(self._steer_cache, 1)
33
34     self._vehicle.apply_control(self._control)

```

Three behaviours emerge from the code. First, when the user presses the up arrow, the throttle is *ramped* from the current value towards the maximum at a rate of 0.01 per tick, instead of being instantaneously applied. This produces the same kind of progressive acceleration that one would expect from a real pedal. Second, when the user presses the down arrow at low speed, the gear is automatically switched to reverse and the throttle is applied in the opposite direction, so that a single key controls both braking and reverse driving. Third, when no steering key is pressed, the steering cache is multiplied by 0.2 at every tick, which produces a smooth return of the wheel to the centre position over a few ticks. This emulates the self-aligning torque of a real steering rack and avoids the on/off behaviour that would result from a binary key state.

The `K_RETURN` key disengages the autopilot through `vehicle.set_autopilot(False)`. After this point, all the commands sent by the LKA

function and by the manual override are accepted by the vehicle, and the driver can resume manual control by simply pressing the directional keys. A `K_TAB` event, handled in the main loop, switches the ego vehicle to a different actor in the fleet, re-spawning the camera in the process; this is convenient during debugging because it allows the operator to inspect the scenario from several points of view without restarting the simulation.

3.7 The Main Control Loop

All the building blocks introduced above are assembled in a single loop that is the hot path of the program. A condensed view of the loop is given below; comments mark the position of each functional block.

```
1  data = pd.read_csv("carla_data.csv", index_col=0)
2
3  crashed = False
4  while not crashed:
5      # 1. Advance the synchronous simulation
6      world.tick()
7
8      # 2. Compute the lateral offset and the steering command
9      lane_shift = lane_shift_calculator(ego_vehicle, data)
10     steer = pid_controller(lane_shift, Kp=1.0, Ki=0.01, Kd=0.1)
11
12     # 3. Render the camera frame
13     gameDisplay.blit(renderObject.surface, (0, 0))
14     pygame.display.flip()
15
16     # 4. Apply the manual override (and possibly overwrite step 2)
17     controlObject.process_control()
18
19     # 5. Process keyboard events
20     for event in pygame.event.get():
21         if event.type == pygame.QUIT:
22             crashed = True
23         controlObject.parse_control(event)
24         if event.type == pygame.KEYUP and event.key == pygame.K_TAB:
25             ego_vehicle.set_autopilot(True)
26             ego_vehicle = random.choice(vehicles)
27             if ego_vehicle.is_alive:
28                 camera.stop(); camera.destroy()
```

```

29         controlObject = ControlObject(ego_vehicle)
30         camera = world.spawn_actor(camera_bp,
        camera_init_trans,
31                                     attach_to=ego_vehicle)
32         camera.listen(lambda image: pygame_callback(image,
        renderObject))

```

The order of the five blocks is not arbitrary. The simulation is advanced first, so that all the state queries that follow are consistent with the same world snapshot. The lateral offset and the steering command are computed second, before any rendering or event handling, in order to minimise the latency between the measurement of the vehicle pose and the application of the corresponding command. The rendering is performed third; because PyGame uses double buffering, the call to `pygame.display.flip()` actually shows the frame produced at the *previous* tick, which is acceptable as long as the operator does not need pixel-accurate feedback. The manual override is applied fourth, so that any keyboard event from the previous tick is honoured before the next tick of the simulator. Finally, the keyboard events are collected and parsed at the end of the loop, in preparation for the next iteration.

A subtle point concerns the interaction between the LKA controller and the manual override. In the current implementation, the value computed by `pid_controller` is *not* written into the `controlObject._control.steer` field; the function returns the value but the loop does not propagate it. This is intentional: the LKA function is intended to be used in two distinct modes, an instrumentation mode in which the steering command is logged for offline analysis, and a closed-loop mode in which the same value is applied to the vehicle. Switching between the two modes is then a matter of adding or removing a single line at the end of step 2, without modifying any of the surrounding code.

3.8 Reference: CARLA’s Built-in Local Planner

The CARLA project ships, as part of its `agents.navigation` module, a reference implementation of a waypoint-following local planner. The relevant source file, `local_planner.py`, was used during the development of this thesis as a comparison point, and a number of design decisions in the controller described above were informed by the corresponding choices in the framework. A short overview of the reference module is therefore given here.

The `LocalPlanner` class wraps a `VehiclePIDController`, instantiates two PID controllers—one for the lateral channel and one for the longitudinal one—and consumes a queue of `(carla.Waypoint, RoadOption)` pairs. At every step, the planner pops obsolete waypoints from the front of the queue, requests the controller to track the next active

waypoint, and refills the queue from the topology of the map when its length falls below a threshold. The default configuration uses

```

1  self._dt = 1.0 / 20.0
2  self._target_speed = 20.0 # km/h
3  self._sampling_radius = 2.0
4  self._args_lateral_dict = {'K_P': 1.95, 'K_I': 0.05, 'K_D': 0.2,
    'dt': self._dt}
5  self._args_longitudinal_dict = {'K_P': 1.0, 'K_I': 0.05, 'K_D': 0.0,
    'dt': self._dt}
6  self._max_throt = 0.75
7  self._max_brake = 0.30
8  self._max_steer = 0.80
9  self._base_min_distance = 3.0
10 self._distance_ratio = 0.5

```

Three design choices in the reference are worth highlighting. First, the sampling rate is also $T_s = 0.05$ s, which is the value adopted in the present work. Second, the lateral PID is significantly more aggressive than the default configuration of `pid_controller`, with $K_p \approx 2$ and a non-zero integral gain; this is consistent with the fact that the reference planner has to handle generic trajectories produced by the global planner, including sharp turns, while the controller of this thesis focuses on lane keeping and is tuned on a comparatively smooth reference. Third, the maximum steering value is capped at 0.80, slightly below the physical limit of 1.0, in order to leave a margin for transient overshoots. The same cap could be added to the implementation of Section 3.4 by setting `u_max = 0.80` at the module level.

The waypoint-management strategy is also different. The reference planner uses a `deque` of fixed maximum length and a distance threshold that grows linearly with the speed of the vehicle:

```

1  self._min_distance = self._base_min_distance + self._distance_ratio *
    vehicle_speed

```

so that a fast-moving vehicle pops waypoints earlier than a slow-moving one. This is more sophisticated than the constant `future_horizon = 3` used in `lane_shift_calculator` and would be a natural extension of the controller; replacing the fixed horizon with a speed-dependent one is the kind of incremental improvement that the modular structure of the implementation is meant to support.

Finally, when the local planner reaches an intersection where multiple successor waypoints are available, the reference module disambiguates them through the `_retrieve_options` helper, which classifies each option as STRAIGHT, LEFT or RIGHT based on the yaw difference with the current waypoint and a 35° threshold:

```

1  def _compute_connection(current_waypoint, next_waypoint, thresh-
    old=35):

```

```

2     n = next_waypoint.transform.rotation.yaw % 360.0
3     c = current_waypoint.transform.rotation.yaw % 360.0
4     diff_angle = (n - c) % 180.0
5     if diff_angle < threshold or diff_angle > (180 - threshold):
6         return RoadOption.STRAIGHT
7     elif diff_angle > 90.0:
8         return RoadOption.LEFT
9     else:
10        return RoadOption.RIGHT

```

The corresponding `RoadOption` enum is then attached to each waypoint in the queue and exposed to higher layers, which can use it to take routing decisions. The controller of this thesis does not rely on this mechanism because the reference trajectory is recorded once at design time, but the same `RoadOption` would be required if the controller were to be combined with a global planner.

In summary, the proposed implementation reuses the same overall architecture as the CARLA reference local planner—synchronous mode, 20 Hz PID, waypoint-based reference—but reduces it to the minimum amount of code that is necessary for the lane keeping function. This makes the controller easier to read and to modify, at the price of a less sophisticated waypoint management. The next chapter will quantify the consequences of this trade-off through a series of closed-loop experiments.

Bibliography

- [1] “Waymo - Self-Driving Cars - Autonomous vehicles - Ride-Hail.” [Online]. Available: <https://waymo.com/>
- [2] A. Kumar and S. Palaniswamy, “Steering Angle Estimation for Self-driving Car Using Deep Learning,” 2020, pp. 196–207. doi: 10.1007/978-981-15-4301-2_17.
- [3] A. R. Plummer, “Model-in-the-Loop testing,” *Proceedings of the Institution of Mechanical Engineers Part I Journal of Systems and Control Engineering*, vol. 220, no. 3, pp. 183–199, May 2006, doi: 10.1243/09596518jsce207.
- [4] S. Demers, P. Gopalakrishnan, and L. Kant, “A Generic Solution to Software-in-the-Loop,” in *MILCOM 2007 - IEEE Military Communications Conference*, 2007, pp. 1–6. doi: 10.1109/MILCOM.2007.4455268.
- [5] H. K. Fathy, Z. S. Filipi, J. Hagena, and J. L. Stein, “Review of hardware-in-the-loop simulation and its prospects in the automotive area,” *Proceedings of SPIE, the International Society for Optical Engineering/Proceedings of SPIE*, May 2006, doi: 10.1117/12.667794.
- [6] M. Abboush, C. Knieke, and A. Rausch, “Hardware-in-the-Loop-Based Real-Time Fault Injection Framework for Dynamic Behavior Analysis of Automotive Software Systems,” *Sensors*, vol. 22, p. 24, 2022, doi: 10.3390/s22041360.
- [7] Admin, “COMPACT Simulator.” [Online]. Available: http://tenergysoft.com/?page_id=13571&utm_source=AdsDzr.ru
- [8] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: an open urban driving simulator.” [Online]. Available: <http://arxiv.org/abs/1711.03938>
- [9] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “AiRSIM: High-Fidelity visual and Physical simulation for autonomous vehicles.” [Online]. Available: <http://arxiv.org/abs/1705.05065>
- [10] C. Gulino *et al.*, “Waymax: An Accelerated, Data-Driven Simulator for Large-Scale Autonomous Driving Research.” [Online]. Available: <https://arxiv.org/abs/2310.08710>
- [11] G. Rong *et al.*, “LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving.” [Online]. Available: <https://arxiv.org/abs/2005.03778>
- [12] M. Müller, “Sim4CV.” [Online]. Available: <https://sim4cv.org/>
- [13] C. Team, “CARLA.” [Online]. Available: <https://carla.org/>
- [14] E. Coelingh, A. Eidehall, and M. Bengtsson, “Collision Warning with Full Auto Brake and Pedestrian Detection - a practical example of Automatic Emergency Braking,” in *13th International IEEE Conference on Intelligent Transportation Systems*, Sept. 2010, pp. 155–160. doi: 10.1109/ITSC.2010.5625077.

- [15] M.-Y. Fu and Y.-S. Huang, “A survey of traffic sign recognition,” *International Conference on Wavelet Analysis and Pattern Recognition*, pp. 119–124, 2010, doi: 10.1109/icwapr.2010.5576425.
- [16] L. Küpper and J. Schug, “Active Night Vision systems,” *SAE technical papers on CD-ROM/SAE technical paper series*, 2002, doi: 10.4271/2002-01-0013.
- [17] W. contributors, “Fatigue detection software.” [Online]. Available: https://en.wikipedia.org/wiki/Fatigue_detection_software
- [18] G. Liu, M. Zhou, L. Wang, H. Wang, and X. Guo, “A blind spot detection and warning system based on millimeter wave radar for driver assistance,” *Optik*, vol. 135, pp. 353–365, 2017, doi: 10.1016/j.ijleo.2017.01.058.
- [19] Y. Song and C. Liao, “Analysis and review of state-of-the-art automatic parking assist system,” in *2016 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, 2016, pp. 1–6. doi: 10.1109/ICVES.2016.7548171.
- [20] M. C. D. What, “Adaptive cruise control - My car does what.” [Online]. Available: <https://mycardoeswhat.org/deeper-learning/adaptive-cruise-control/>
- [21] W. Son, Y. Ha, T. Oh, S. Woo, S. Cho, and J. Yoo, “PG-Based Vehicle-In-the-Loop Simulation for System Development and Consistency Validation,” *Electronics*, vol. 11, no. 24, 2022, doi: 10.3390/electronics11244073.
- [22] J. Cheng, Z. Wang, X. Zhao, Z. Xu, M. Ding, and K. Takeda, “A Survey on Testbench-Based Vehicle-in-the-Loop Simulation Testing for Autonomous Vehicles: Architecture, Principle, and Equipment,” *Advanced Intelligent Systems*, vol. 6, no. 6, p. 2300778, 2024, doi: <https://doi.org/10.1002/aisy.202300778>.
- [23] X. Hu, J. Zeng, S. Li, W. Liao, X. Ding, and Y. Ren, “Vehicle-in-loop simulation test research for autonomous vehicle,” in *Eighth International Conference on Electro-mechanical Control Technology and Transportation (ICECTT 2023)*, S. Easa and W. Wei, Eds., SPIE, 2023, p. 127900T. doi: 10.1117/12.2690076.
- [24] Y. Kebbati, N. Ait-Oufroukh, D. Ichalal, and V. Vigneron, “Lateral control for autonomous wheeled vehicles: A technical review,” *Asian Journal of Control*, vol. 25, no. 4, pp. 2539–2563, 2023, doi: <https://doi.org/10.1002/asjc.2980>.
- [25] A. Artuñedo, M. Moreno-Gonzalez, and J. Villagra, “Lateral control for autonomous vehicles: A comparative evaluation,” *Annual Reviews in Control*, vol. 57, p. 100910, 2024, doi: <https://doi.org/10.1016/j.arcontrol.2023.100910>.
- [26] S. Dominguez, A. Ali, G. Garcia, and P. Martinet, “Comparison of lateral controllers for autonomous vehicle: Experimental results,” in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, 2016, pp. 1418–1423. doi: 10.1109/ITSC.2016.7795743.
- [27] J. M. Snider and others, “Automatic steering methods for autonomous automobile path tracking,” *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RITR-09-08*, 2009.

- [28] W. contributors, “Unreal Engine 4.” [Online]. Available: https://en.wikipedia.org/wiki/Unreal_Engine_4

List of Figures

Figure 1.1 The historical development of autonomous driving [2].	4
Figure 1.2 MIL, SIL , PIL, HIL and VIL tests in V-cycle development process [6].	5
Figure 1.3 Driving Test Simulator [7]	6
Figure 1.4 Comparing Waymo Simulator graphics and the real world.	7
Figure 1.5 SVL Simulator Screen	8
Figure 1.6 Sim4CV: A Photo-Realistic Simulator for Computer Vision Applications	8
Figure 1.7 Carla Simulator Screen	9
Figure 1.8 Automatic Emergency Braking system.	10
Figure 1.9 Traffic Sign Recognition System	10
Figure 1.10 Night Vision	11
Figure 1.11 Blind Spot Warning System	11
Figure 1.12 Parking Asistant	11
Figure 1.13 Automatic Headlight Control	12
Figure 1.14 Lane Keeping Asistant	12
Figure 1.15 Adaptive Cruise Control	12
Figure 2.1 Single Track Model	17
Figure 2.2 Vehicle Reference System	18
Figure 2.3 Friction Coefficents in different conditions	19
Figure 3.1 Block diagram of the proposed Lane Keeping Assist co-simulation. The CARLA server holds the world state; the client computes the steering command from the lane reference and the vehicle pose, and applies it through <code>vehicle.apply_control</code>	28
Figure 3.2 Geometric definition of the lane shift d as the signed perpendicular distance between the vehicle position P_v and the line through the closest waypoint P_w in the direction of the lane forward vector $\hat{t}$	31

List of Tables

Table 1.1 Comparative Feature Scoring of Simulators	9
Table 3.2 Qualitative effect of each PID gain on the closed-loop response of a generic plant.	29

Appendix A

Carla and Pycharm IDE Interfacing

To control and communicate with Carla, the Python API is used. Therefore, it is necessary to have a Python version compatible with the Carla version we plan to use. However, updates to Carla often require updating the Python version at regular intervals. This makes it mandatory to reconcile Carla and Python with each new update. However, by taking advantage of uv's ability to use different Python versions through different digital environments, we can solve this problem. Thus, we can have multiple environments and Carla models on the same computer, which also reduces the burden of this process. The steps for setting up Carla-uv communication are as follows:

- Downloaded Carla 0.9.16(UE4)
- Installed Pycharm IDE
- Installed uv python package and project manager:

```
1  set PROJECT=carla_lane
2  powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
   0.10.8/install.ps1 | iex"
3  uv clean
4  git clone https://github.com/axmud/lane_assist.git %PROJECT%
5  uv init %PROJECT% -p "==3.10.20"
6  cd %PROJECT%
7  uv add setuptools==66.1.1
8  uv add wheel==0.46.3
9  uv add alumentations==0.4.6 --no-build-isolation
10 uv add --requirements requirements.txt
```
- Pytorch requirements. We need to add these lines to the end of pyproject.toml file in the project folder:

```
1  cu121 = ["torch==2.3.1", "torchvision==0.18.1", "torchau-
   dio==2.3.1"]
2  [tool.uv.sources]
3  torch = [
4    { index = "pytorch-cu121", extra = "cu121" }
5  ]
```

```

6  torchvision = [
7      { index = "pytorch-cu121", extra = "cu121" }
8  ]
9  torchaudio = [
10     { index = "pytorch-cu121", extra = "cu121" }
11 ]
12 [[tool.uv.index]]
13 name = "pytorch-cu121"
14 url = "https://download.pytorch.org/whl/cu121"
15 explicit = true

```

- Next, we will run this:

```
1  uv sync --extra cu121
```

- Dependency adjustments:

```

1  uv pip install --no-build-isolation -e .
2  uv pip install numpy==1.23.1
3  uv pip install hydra-core
4  uv pip uninstall setuptools
5  uv pip install setuptools==66.1.1
6  uv pip install carla==0.9.16

```